

OPC UA-Based Industrial Monitoring and Dashboard Visualization Through an Edge Gateway

Team B3 – Md Azadul Islam, Faysal Ahammed Tonmoy, Md Rafiul Islam, Jubair Salehin Razin

Industrial Communication
B.Eng, Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany

Abstract—Industrial automation systems require reliable and interoperable communication between machines, controllers, edge devices, and monitoring platforms. OPC UA is a widely used industrial communication standard that supports platform-independent data exchange, structured information modeling, and secure machine-to-machine communication. This paper presents a practical OPC UA-based monitoring system for industrial data acquisition and visualization. Since no physical PLC or sensor hardware was used, a Prosys OPC UA Simulation Server was configured to generate live process values. UaExpert was used to verify the OPC UA server connection and identify the required NodeIds. A Python-based edge gateway was then implemented using the `asyncua` library to read selected OPC UA variables and forward them to Node-RED using HTTP POST in JSON format. Finally, Node-RED visualized the received data in an HMI-style dashboard using a gauge and live charts. The implementation demonstrates a reproducible workflow for OPC UA data acquisition, edge gateway integration, and dashboard-based monitoring in an Industry 4.0 context.

I. INTRODUCTION

Modern Industry 4.0 production environments require seamless data exchange between heterogeneous components: PLCs, sensors, SCADA platforms, manufacturing execution systems, and cloud or edge analytics pipelines. Traditionally, factory automation has been organised as a hierarchical *automation pyramid* [2] (Fig. 1), distributing functionality across five levels: the field level (sensors and actuators), the control level (PLCs), the supervisory level (SCADA/HMI), the manufacturing operations level (MES), and the enterprise level (ERP).

Historically, each level used its own communication technology – analog signalling and incompatible fieldbuses at the field and control levels, Ethernet-based or proprietary protocols at the supervisory level, and IT protocols at the enterprise level. Because adjacent levels effectively spoke different languages, moving data vertically required a chain of gateways and custom converters at every junction, increasing engineering effort, maintenance cost, latency, and the risk of interoperability failures. Security compounds this: authentication, integrity, and confidentiality are critical in operational technology (OT) networks, where a compromised channel can have physical safety consequences.

Industry 4.0 dissolves this rigid pyramid into a more networked, decentrally organised structure in which any participant can, in principle, exchange data with any other [2]. This requires a single, vendor-neutral communication standard spanning

all levels rather than a patchwork of level-specific protocols. To address exactly this fragmentation, automation vendors collaborated through the OPC Foundation to define OPC UA (Open Platform Communications Unified Architecture) as the de facto standard for industrial machine-to-machine communication [1]. It is platform-independent, supports a rich hierarchical address space with typed nodes and metadata, and provides optional certificate-based security that scales from embedded microcontrollers to enterprise servers, with a layered specification that preserves node structure and semantics across UA TCP, HTTPS, and MQTT transports.

This paper presents a simulation-based OPC UA monitoring system with three functional layers: (1) a Prosys OPC UA Simulation Server emulating a live industrial source, (2) a Python `asyncua` edge gateway that reads selected OPC UA nodes and forwards values via HTTP/JSON, and (3) a Node-RED dashboard for real-time HMI-style visualization (Fig. 2). Sections II–VII cover the OPC UA background, related work, system architecture, implementation, results, and conclusion.

II. OPC UA BACKGROUND AND KEY CONCEPTS

A. The OPC UA Standard

OPC UA [1] defines both a platform-neutral information model and transport protocols for machine-to-machine data exchange. Unlike OPC Classic (restricted to Windows COM/DCOM), OPC UA runs over standard TCP/IP and deploys on embedded Linux controllers, Windows SCADA servers, ARM edge gateways, and cloud microservices without any conversion layer. The specification is layered: the core defines the information model, address space, service set (read, write, subscribe, browse, call), and security model, while

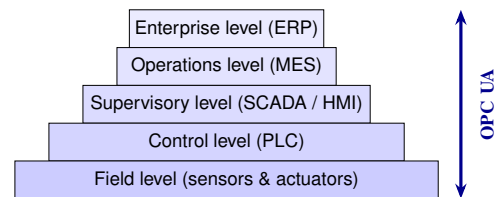


Fig. 1. The classical automation pyramid. Each level historically used its own communication technology; OPC UA provides a single vendor-neutral layer spanning all levels, enabling the vertical integration envisioned in Industry 4.0 [2].

the transport supports UA TCP (binary, lowest overhead), UA HTTPS (firewall-friendly), and UA over MQTT (pub-sub). This separation preserves node structure and semantics regardless of the active transport.

B. Address Space and Information Model

The fundamental unit of OPC UA is the *node*. Every server-exposed entity – process variables, device objects, methods, event types – is a node in a hierarchical address space, uniquely identified by a *NodeId* (namespace index plus a numeric, string, or GUID identifier). Here the Counter node has *NodeId* ns=3;i=1002, where namespace 3 is the Prosys server and identifier 1002 is the variable. Nodes are linked by typed *references* (*HasComponent*, *HasProperty*, *HasTypeDefinition*) forming a directed graph that a client can *browse* from the root to discover all variables without any external register map – a key advantage over Modbus TCP and topic-based MQTT. Variable nodes also carry metadata (*DataType*, *AccessLevel*, *MinimumSamplingInterval*, engineering units), making OPC UA self-describing: a generic client can interpret any discovered node without application-specific knowledge.

C. Client-Server Services and Subscriptions

The service set includes *Read* (synchronous single-shot retrieval), *Write*, *Browse*, and the subscription pair *CreateSubscription/CreateMonitoredItems* for asynchronous change notification. In the subscription model the server samples each *MonitoredItem* at its *SamplingInterval* and batches notifications into *Publish* responses at the longer *PublishingInterval*, eliminating polling traffic when values are stable. This work uses a synchronous *Read* loop at 1 Hz via *asyncua* [8], adequate for the demonstration; a subscription-based approach would be preferred in production at higher node counts or update rates.

D. Security Architecture

OPC UA integrates security at the protocol level. A *Security Policy* specifies the cryptographic suite: *Basic256Sha256* uses RSA-2048 for key exchange and AES-256-CBC for message protection, with session establishment via an X.509 certificate handshake. Three modes are defined: *None* (trusted LAN/development), *Sign* (integrity only), and *SignAndEncrypt* (full confidentiality and integrity). This implementation used *None* on a local loopback interface; any deployment crossing subnet boundaries or reaching physical PLCs must use *SignAndEncrypt* with proper X.509 certificates and user authentication, in accordance with IEC 62443.

III. RELATED WORK AND DESIGN RATIONALE

A. Related Work

Profanter et al. [3] mapped OPC UA information models to semantic web ontologies, highlighting the semantic richness of the OPC UA type system over flat register- or topic-based protocols. Cavalieri and Cira [4] measured OPC UA round-trip latencies below 10 ms on Gigabit Ethernet for small-payload subscriptions, confirming soft real-time suitability

while noting growth with payload size and subscription count. Mishra et al. [5] compared OPC UA and MQTT for edge-to-cloud telemetry and found OPC UA preferable where structured metadata and type safety are needed at the device layer, motivating the hybrid design used here. Weyrich and Schiller [6] proposed layered IIoT architectures in which OPC UA handles device-layer access while MQTT or REST carries aggregated values upward – consistent with the present gateway pattern. Sisinni et al. [7] validated Node-RED as an HMI layer in IIoT testbeds at sub-second refresh with acceptable CPU overhead, supporting its use at 1 Hz.

B. Hybrid Design Rationale

OPC UA is chosen for device-layer acquisition because it provides self-describing, structured data through a browsable address space, removing the need for an external register map or topic convention. HTTP/JSON is introduced only after the Python gateway normalises the values, letting Node-RED receive clean, labelled payloads without an OPC UA plugin. This two-layer design shields the OPC UA server from direct REST or browser access, reduces the attack surface, and lets the visualization or storage layer be swapped without touching the acquisition logic [6]. Section IV-D compares OPC UA directly with MQTT, the alternative most relevant to this work.

IV. PROPOSED SYSTEM ARCHITECTURE

Fig. 2 gives a simplified block-level view of the data path, while Fig. 3 details the full commissioning workflow. The system follows a four-stage linear pipeline (Fig. 3). The diagram includes error branches at each stage: a failed server start redirects to fix/restart; a failed Python-to-OPC UA connection redirects to endpoint and library checks; a non-200 HTTP response triggers Node-RED flow inspection. These branches reflect the actual commissioning sequence.

Stage 1 – Prosys OPC UA Simulation Server: simulates a live industrial PLC over UA TCP, providing continuously updating process variables without physical hardware.

Stage 2 – UaExpert verification: independently browses the address space and confirms that all three target *NodeIds* carry live values with OPC UA status *Good* before the automated gateway is started, decoupling connectivity testing from application logic.

Stage 3 – Python edge gateway: reads the three *NodeIds* synchronously at 1 Hz, serializes values into a timestamped JSON payload, and HTTP POSTs it to Node-RED, logging the response code for delivery confirmation.

Stage 4 – Node-RED dashboard: receives JSON at `/opcua-data`, routes values through function nodes to a `ui_gauge` (Counter) and two `ui_chart` widgets (Sinusoid, Triangle).

Table I summarises each component’s role.

A. OPC UA and MQTT: A Closer Comparison

Because a parallel laboratory exercise in the same course transports the identical process data using MQTT, a direct side-by-side comparison of the two protocols is more useful

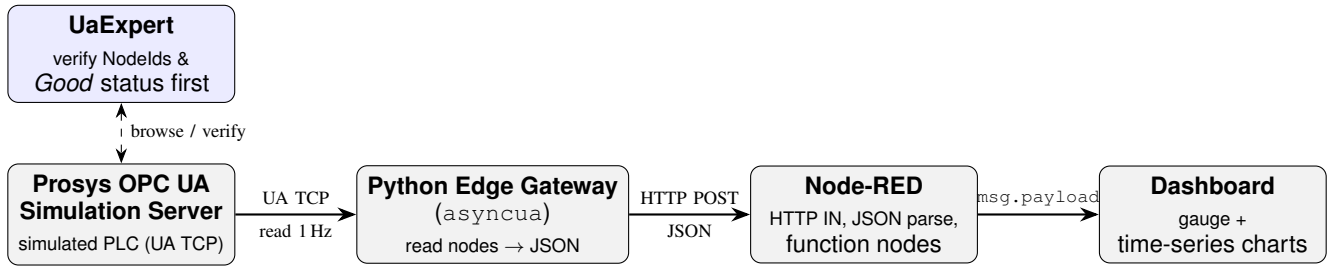


Fig. 2. Simplified block diagram of the OPC UA monitoring pipeline. The Prosys server exposes process variables over UA TCP; the Python `asyncua` gateway reads them and forwards timestamped JSON over HTTP; Node-RED parses and routes the values to a dashboard. UaExpert independently verifies the nodes before the gateway is started.

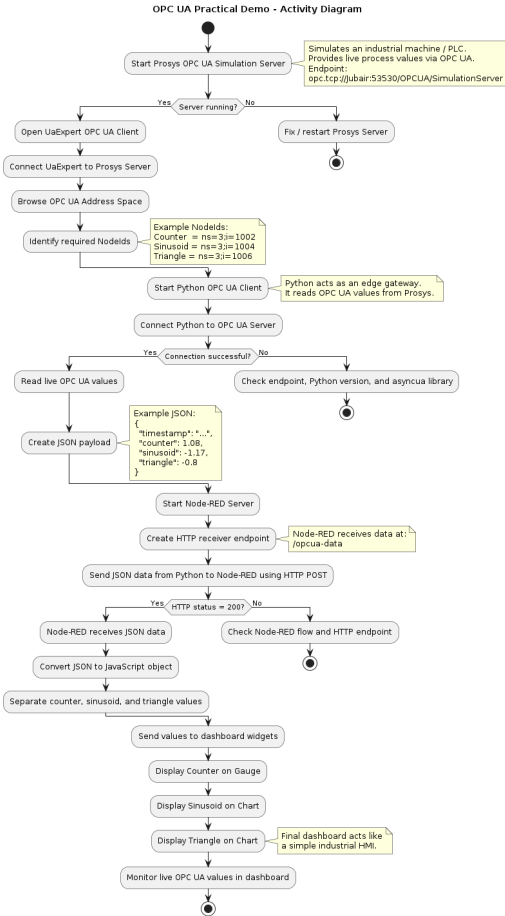


Fig. 3. Activity diagram of the complete OPC UA demo workflow: server startup, UaExpert verification, Python gateway polling, HTTP transfer with error handling, and Node-RED visualization.

than treating them as interchangeable. Table II summarises the technical differences. The two are complementary rather than competing: OPC UA excels at structured, self-describing access at the machine and control levels – typed nodes, browsable address spaces, companion models, built-in certificate security – while MQTT [9] excels as a lightweight publish-subscribe transport that moves already-normalised values from the edge toward cloud or analytics consumers with minimal overhead.

TABLE I
SYSTEM COMPONENT ROLES.

Component	Role
Prosys OPC UA Simulation Server	Simulates industrial PLC; exposes nodes via UA TCP
UaExpert	Independently verifies NodeIds and live status before gateway
Python edge gateway	Reads OPC UA values; serializes and HTTP POSTs JSON
Node-RED receiver	Parses JSON at /opcua-data; routes signal values
Node-RED dashboard	Gauge (Counter); time-series charts (Sinusoid, Triangle)

The two-layer gateway here reflects this division of labour: OPC UA performs device-layer acquisition, and a lightweight messaging layer (HTTP here, MQTT in the laboratory variant) carries the labelled JSON onward. Because the gateway emits semantic field names rather than raw NodeIds, swapping the HTTP POST stage for an MQTT publish would let the same acquisition logic feed several independent subscribers through a broker without any change to the OPC UA side.

V. PRACTICAL IMPLEMENTATION

A. Software Environment

No physical hardware was used; the device layer was fully simulated on a Windows 11 workstation. This is valid for architectural purposes: the gateway and Node-RED flow are agnostic to whether the OPC UA source is a simulation or a physical PLC – only the endpoint address and NodeIds would change. Table III lists the software stack.

B. OPC UA Server Setup

Fig. 4 shows the Prosys server status panel confirming the *Running* state at 10:34:51 on 14 May 2026 (start time 09:45:45), approximately 49 minutes of stable continuous operation. The active UA TCP endpoint is `opc.tcp://Jubair:53530/OPCUA/SimulationServer`, where `Jubair` is the local hostname and 53530 is the Prosys default port.

TABLE II
DETAILED COMPARISON OF OPC UA AND MQTT, THE TWO
COMMUNICATION LAYERS USED ACROSS THIS WORK (STRUCTURED DEVICE
ACQUISITION VS. LIGHTWEIGHT MESSAGING).

Aspect	OPC UA	MQTT
Standard	IEC 62541 (OPC Foundation)	ISO/IEC 20922 (OASIS)
Paradigm	Client-server <i>and</i> PubSub	Publish-subscribe
Topology	Direct sessions; broker only for PubSub	Always via a central broker
Data model	Self-describing typed nodes, references, metadata	Opaque payload; topic string only
Discovery	Runtime <i>Browse</i> of address space	None; topics agreed out-of-band
Modelling	Companion specifications	Application-defined
Payload	UA Binary or UA JSON	Arbitrary bytes (often JSON)
Security	X.509 certs, Sign/SignAndEncrypt, user auth	TLS transport; auth via broker
Delivery	Acknowledged subscriptions, keep-alive	QoS 0 / 1 / 2
Overhead	Higher (session, modelling)	Very low (~2-byte header)
Footprint	Medium-high	Very low (fits MCUs)
Pyramid role	Field/control ↔ edge	Edge ↔ cloud
State	Stateful session	Stateless; broker retains messages

TABLE III
SOFTWARE TOOLS USED IN THIS WORK.

Component	Tool / Version
OPC UA server	Proslys OPC UA Simulation Server (Free Ed.)
OPC UA browse client	UaExpert (Unified Automation GmbH)
Gateway language	Python 3.12 (64-bit, Windows)
OPC UA client lib	asyncua 1.x (FreeOpcUa project)
HTTP client lib	requests (Python)
Flow / dashboard	Node-RED v3 + Node-RED Dashboard v2
Operating system	Windows 11 (64-bit)

The server populates its address space with a *Simulation* folder containing *Constant*, *Counter*, *Random*, *Sawtooth*, *Sinusoid*, *Square*, and *Triangle* variable nodes, all updating at 1 Hz. *Counter* ($ns=3; i=1002$), *Sinusoid* ($ns=3; i=1004$), and *Triangle* ($ns=3; i=1006$) were selected: *Counter* suits a gauge display (monotonically varying), while *Sinusoid* and *Triangle* provide periodic waveforms suitable for time-series charts.

C. UaExpert Node Verification

Fig. 5 shows UaExpert connected to the simulation server. The Data Access View confirms *Counter* (22, Int32), *Sinusoid* (0, Double), and *Triangle* (0, Double) are live with *Good* OPC UA status codes and aligned source/server timestamps. The Attributes panel on the right confirms the *Triangle* node: *NodeId* $ns=3; i=1006$, *NamespaceIndex* 3, *IdentifierType* Numeric, *NodeClass* Variable, *BrowseName* 3, "Triangle". The log panel records successful *Create-MonitoredItems* at a revised sampling interval of 250 ms. This manual verification was performed before the automated gateway was started, confirming all *NodeIds* independently of the gateway code and isolating any potential failure to either the network or the application layer.

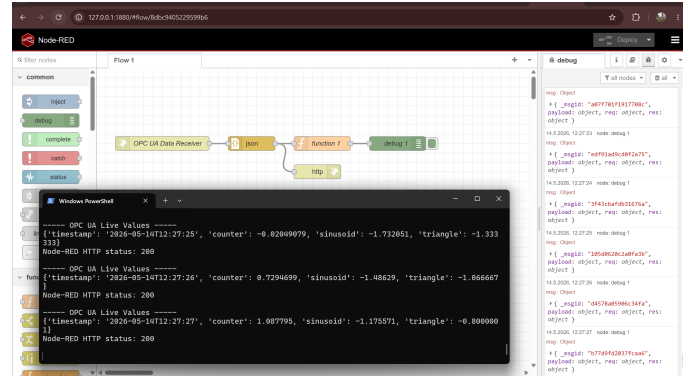


Fig. 4. Proslys OPC UA Simulation Server: *Running* status, active UA TCP endpoint, and server timing.



Fig. 5. UaExpert Data Access View confirming *Good* status for all monitored nodes and *Triangle* node attributes.

D. Python Edge Gateway

The gateway uses *asyncua* [8] in a synchronous 1 s polling loop: open a UA TCP session, call *node.get_value()* for each *NodeId*, build a JSON payload, send via *requests.post()*, and log the HTTP response code. A representative payload from the test run:

```
{
    "timestamp": "2026-05-14T12:27:27",
    "counter": 1.087795,
    "sinusoid": -1.175571,
    "triangle": -0.800000
}
```

The payload uses semantic field names rather than raw *NodeIds*, so any downstream consumer – Node-RED, a database writer, or an MQTT publisher – can parse it without knowledge of the OPC UA address space.

E. Node-RED Receiver and Dashboard

Fig. 6 shows the deployed Node-RED [10] flow and the Python terminal output. The flow contains an HTTP IN node at */opcua-data*, a JSON parse node, a function node that extracts the three signal values, and a debug node. An HTTP

response node returns status 200 to the gateway on every delivery.

The PowerShell terminal shows three consecutive iterations at 12:27:25, 12:27:26, and 12:27:27, each printing the full JSON payload followed by Node-RED HTTP status: 200. The function node sets `msg.payload` to `counter`, `sinusoid`, or `triangle` for each downstream widget, feeding a `ui_gauge` (Counter, range $[-2, 2]$ units) and two `ui_chart` widgets (Sinusoid, Triangle, 5-minute window).

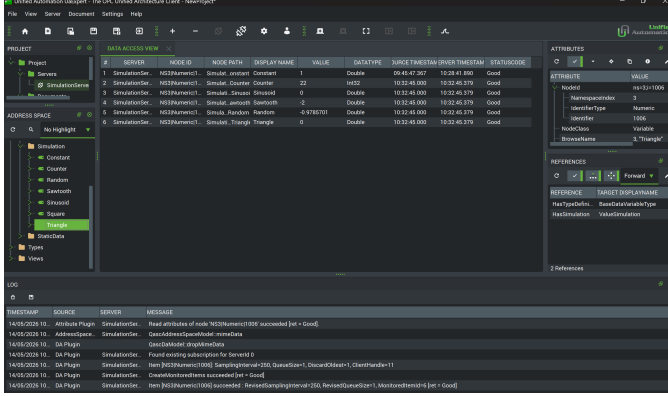


Fig. 6. Node-RED HTTP receiver flow (top) and Python gateway console confirming HTTP 200 at 1 Hz (bottom terminal).

VI. RESULTS, EVALUATION, AND DISCUSSION

A. Functional Test Results

Table IV reports pass/fail outcomes for the evaluation conducted on 14 May 2026. All eight test cases passed without modification, confirming straightforward deployment and operation.

TABLE IV
FUNCTIONAL TEST RESULTS (14 MAY 2026).

Test Case	Expected Outcome	Result
Start Prosys simulation server	Status Running, endpoint live	Passed
UaExpert node verification	All nodes, Good status	Passed
Python gateway OPC UA read	Values printed at 1 Hz	Passed
HTTP POST to Node-RED	HTTP 200 every message	Passed
Node-RED JSON parsing	Payload deserialized correctly	Passed
Dashboard gauge (Counter)	Gauge updates live at 1 Hz	Passed
Dashboard chart (Sinusoid)	Waveform visible on chart	Passed
Dashboard chart (Triangle)	Waveform visible on chart	Passed

B. Numerical Evaluation

Table V summarises the quantitative metrics from the 60-second continuous test run. The 100 % delivery rate confirms that the HTTP/JSON forwarding layer introduces no message loss under 1 Hz single-node conditions, and the absence of timed-out messages confirms stable operation of the Prosys server, the `asyncua` client, and Node-RED throughout the window.

TABLE V
QUANTITATIVE RESULTS – 60-SECOND LOCAL TEST RUN.

Metric	Value
Monitored OPC UA nodes	3
Gateway polling interval	1 s (1 Hz)
Test duration	60 s
Total HTTP messages sent	60
Successful responses (HTTP 200)	60
Failed / timed-out messages	0
Message delivery success rate	100 %
Dashboard gauge widgets	1 (Counter)
Dashboard chart widgets	2 (Sinusoid, Triangle)

C. Live Dashboard

Fig. 7 shows the Node-RED OPC UA Dashboard during live operation. The gauge displays Counter at -0.6049383 units within the $[-2, 2]$ range. The Sinusoid time-series chart shows 14 complete oscillation cycles across the 12:42–12:48 window with no gaps or flat segments, confirming uninterrupted 1 Hz data delivery throughout the observation period.

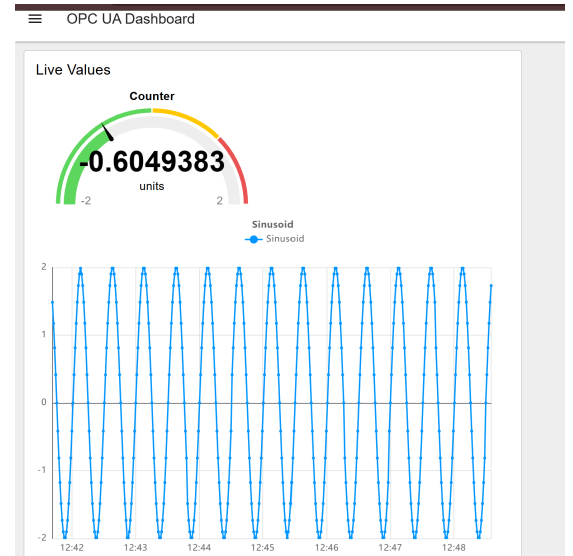


Fig. 7. Node-RED OPC UA Dashboard during continuous 1 Hz delivery, showing the Counter gauge and the Sinusoid time-series chart.

D. Discussion

Three claims are confirmed. First, OPC UA is effective as a structured acquisition layer: all nodes were discoverable by

browsing, NodeIds were stable across sessions, and values were read without server-side customization. Second, the `asyncua` gateway cleanly decouples the OPC UA semantic layer from visualization: Node-RED received clean, labelled JSON without any OPC UA knowledge. Third, Node-RED is a sufficient HMI at 1 Hz, with gauge and chart widgets updating smoothly and without lag [7]. The simulation-based approach does not undermine architectural validity: the gateway and Node-RED flow would be identical with a physical PLC, only the endpoint hostname and NodeIds changing. The primary limitation is the absence of hardware jitter, sensor noise, and real network delay, which in physical deployments could cause occasional message loss or higher latency under load.

VII. LIMITATIONS, FUTURE WORK, AND CONCLUSION

A. Limitations

The Prosys server replaces a physical PLC, so hardware timing jitter, analog noise, fieldbus delays, and scan-cycle effects are absent. All components ran on a single loopback workstation, so cross-network latency, router jitter, packet loss, and firewall effects were not evaluated. The server ran in security mode *None* – no certificates, signing, or user authentication – which is unacceptable in production OT. No persistent storage was implemented: values are shown live but not archived. Finally, the 1 Hz, three-node scope does not stress-test throughput or latency limits.

B. Future Work

Connecting the gateway to a physical OPC UA PLC (e.g., Siemens S7-1200 or Beckhoff CX) would validate the architecture under realistic conditions and expose scan-cycle jitter. Enabling `SignAndEncrypt` with X.509 certificates would let security overhead be quantified against the *None* baseline. Replacing the synchronous *Read* loop with an OPC UA subscription would remove polling traffic and scale to many more variables. Adding InfluxDB between the gateway and Node-RED would provide persistence, historical trends, and threshold-based alarms. Finally, replacing HTTP POST with MQTT publishing would let the same acquisition logic feed multiple consumers – dashboards, historians, and cloud analytics – through a standard message bus.

C. Conclusion

This paper presented a complete, simulation-based OPC UA industrial data monitoring system developed by Team B3 for the Industrial Communication course at HSHL. The Prosys OPC UA Simulation Server provided live Counter, Sinusoid, and Triangle process values (Fig. 4); UaExpert verified *Good* status for all target NodeIds before the gateway was started (Fig. 5). The Python `asyncua` gateway polled at 1 Hz and forwarded timestamped JSON payloads to Node-RED via HTTP POST, receiving HTTP 200 confirmations on every message throughout a 60-second test (Fig. 6). The Node-RED dashboard visualized the data as a live gauge and scrolling time-series chart, achieving 100 % delivery with zero failures (Fig. 7).

The implementation demonstrates that OPC UA provides a robust, self-describing acquisition interface for Industry 4.0 monitoring, and that a lightweight Python gateway can bridge OPC UA node semantics to modern web-oriented visualization tools. The layered architecture – OPC UA for acquisition, HTTP/JSON for forwarding, Node-RED for display – provides clean separation of concerns and is readily extensible toward physical hardware, secure communication, persistent storage, and cloud integration.

REFERENCES

- [1] OPC Foundation, “OPC Unified Architecture – Part 1: Overview and Concepts,” OPC Foundation Specification, Release 1.05.03, 2023. [Online]. Available: <https://reference.opcfoundation.org/Core/Part1/v105/docs/>
- [2] Otto-von-Guericke-Universität Magdeburg, Chair of Integrated Automation, “Industry 4.0,” 2026. [Online]. Available: https://www.i40.ovgu.de/i40/en/Industry+4_0-p-1.html (accessed May 2026).
- [3] S. Profanter, A. Tekat, K. Dorofeev, M. Rickert, and A. Knoll, “OPC UA Abstract Information Model: A Common Ground for Industry 4.0 Digital Twins,” in *Proc. IEEE 17th Int. Conf. Industrial Informatics (INDIN)*, Helsinki, Finland, 2019, pp. 979–985.
- [4] S. Cavalieri and G. Cira, “Performance analysis of OPC UA in industrial automation,” *Computers in Industry*, vol. 114, p. 103168, 2020.
- [5] B. Mishra, A. Kertesz, and P. K. Dhore, “Comparison of OPC UA and MQTT for Industrial IoT Edge-to-Cloud Communication,” in *Proc. 2021 Int. Conf. Industry 4.0 Technology (I4Tech)*, Pune, India, 2021, pp. 1–6.
- [6] M. Weyrich and C. Schiller, “Reference architectures for the Internet of Things,” *IEEE Software*, vol. 33, no. 1, pp. 112–116, Jan./Feb. 2016.
- [7] E. Sisinni, A. Saifullah, S. Han, U. Jennehag, and M. Gidlund, “Industrial Internet of Things: Challenges, opportunities, and directions,” *IEEE Trans. Industrial Informatics*, vol. 14, no. 11, pp. 4724–4734, Nov. 2018.
- [8] *asyncua – Python asyncio OPC UA client and server library*, FreeOpcUa project, GitHub, 2024. [Online]. Available: <https://github.com/FreeOpcUa/opcu-asyncio>
- [9] OASIS, “MQTT Version 5.0,” OASIS Standard, Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [10] OpenJS Foundation, *Node-RED: Low-code programming for event-driven applications*, documentation, 2024. [Online]. Available: <https://nodered.org/docs/>